B.Tech. BCSE497J - Project-I

AI-DRIVEN FRAMEWORK FOR DETECTION, RECOMMENDATION AND VERIFIED MIGRATION OF POST-QUANTUM CRYPTOGRAPHY IN PYTHON CODEBASES

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

23BCI0044 ARPIT MITTAL
23BCT0040 PRIYANSHU SEN
23BCT0056 VEDANSH TRIVEDI

Under the Supervision of

BADRINATH N
Associate Professor Grade 2
School of Computer Science and Engineering (SCOPE)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

We hereby declare that the project entitled “AI-Driven Framework for Detection, Recommendation and Verified Migration of Post-Quantum Cryptography in Python Codebases” submitted by us, for the award of the degree of Bachelor of Technology in Computer Science and Engineering to Vellore Institute of Technology, is a record of bonafide work carried out by us under the supervision of BADRINATH N, Associate Professor Grade 2.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date :

Signature of the Candidates

CERTIFICATE

This is to certify that the project entitled "AI-Driven Framework for Detection, Recommendation and Verified Migration of Post-Quantum Cryptography in Python Codebases" submitted by ARPIT MITTAL (23BCI0044), PRIYANSHU SEN (23BCT0040), VEDANSH TRIVEDI (23BCT0056), School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by them under my supervision during Fall Semester 2026-2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in our opinion meets the necessary standards for submission.

Place : Vellore

Date :

Signature of the Guide

Examiner(s)

<<Head of the Department>>

Computer Science and Engineering

ACKNOWLEDGEMENTS

We are deeply grateful to the management of Vellore Institute of Technology for providing us with the opportunity and the facilities to carry out this project. The academic environment and the resources made available to us throughout the semester were instrumental in allowing this work to be taken from an idea to a working system.

We extend our sincere thanks to Dr. Jaisankar N, Dean of the School of Computer Science and Engineering, for the encouragement and the support extended to student research. The emphasis placed on building and evaluating real systems, rather than only studying them, shaped the direction this project took.

We express our appreciation to <<Head of the Department>>, Head of the Department, for valuable guidance and for fostering a collaborative atmosphere in which questions could be raised freely and difficulties discussed openly.

We are especially thankful to our project guide, BADRINATH N, Associate Professor Grade 2, for dedicated mentorship and consistently constructive feedback. The insistence on measuring what we claimed, rather than asserting it, changed the shape of this project for the better: the evaluation reported in Chapter 7 exists because of that guidance, and it uncovered defects in our own implementation that we would otherwise have missed.

Finally, we thank our families and our classmates for their patience and encouragement over the course of the semester.

ARPIT MITTAL,
PRIYANSHU SEN,
VEDANSH TRIVEDI

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	i
	List of Figures	iii
	List of Tables	iv
	List of Abbreviations	v
1.	**INTRODUCTION**	1
	1.1 Background	1
	1.2 Motivations	2
	1.3 Scope of the Project	3
2.	**PROJECT DESCRIPTION AND GOALS**	5
	2.1 Literature Review	5
	2.2 Research Gap	8
	2.3 Objectives	9
	2.4 Problem Statement	10
	2.5 Project Plan	11
3.	**TECHNICAL SPECIFICATION**	12
	3.1 Requirements	12
	3.1.1 Functional	12
	3.1.2 Non-Functional	13
	3.2 Feasibility Study	14
	3.2.1 Technical Feasibility	14
	3.2.2 Economic Feasibility	15
	3.2.3 Social Feasibility	15
	3.3 System Specification	16
	3.3.1 Hardware Specification	16
	3.3.2 Software Specification	16
4.	**DESIGN APPROACH AND DETAILS**	18
	4.1 System Architecture	18
	4.2 Design	19
	4.2.1 Data Flow Diagram	19
	4.2.2 Use Case Diagram	21
	4.2.3 Class Diagram	22
	4.2.4 Sequence Diagram	23
5.	**METHODOLOGY AND TESTING**	24
	5.1 Module Description	24
	5.2 Testing	26
6.	**PROJECT DEMONSTRATION**	28
7.	**RESULT AND DISCUSSION**	30
8.	**CONCLUSION**	36
9.	**REFERENCES**	38
	APPENDIX A – SAMPLE CODE	39

List of Figures

Figure No.	Title	Page No.
4.1	System architecture of the PQC migration framework	18
4.2	Data flow diagram - Level 0 (context diagram)	19
4.3	Data flow diagram - Level 1	20
4.4	Use case diagram	21
4.5	Class diagram of the core data model and engines	22
4.6	Sequence diagram for the end-to-end run command	23
7.1	Classical versus post-quantum key and output sizes	33
7.2	Classical versus post-quantum operation timings	34

List of Tables

Table No.	Title	Page No.
2.1	Summary of reviewed literature and its limitations	7
2.2	Project plan and phase-wise schedule	11
3.1	Functional requirements	12
3.2	Non-functional requirements	13
3.3	Hardware specification	16
3.4	Software specification	16
5.1	Module description summary	25
5.2	Test suite composition	27
7.1	Detection benchmark composition	30
7.2	Detection performance	30
7.3	In-scope detection performance by algorithm family	31
7.4	End-to-end pipeline results on the sample applications	32
7.5	Measured key and output sizes	33
7.6	Measured performance of classical and post-quantum schemes	34

List of Abbreviations

AES Advanced Encryption Standard

API Application Programming Interface

AST Abstract Syntax Tree

CLI Command Line Interface

DER Distinguished Encoding Rules

DFD Data Flow Diagram

DH Diffie-Hellman

ECC Elliptic Curve Cryptography

ECDH Elliptic Curve Diffie-Hellman

ECDSA Elliptic Curve Digital Signature Algorithm

FIPS Federal Information Processing Standard

GCM Galois/Counter Mode

HTML HyperText Markup Language

JSON JavaScript Object Notation

KEM Key Encapsulation Mechanism

ML-DSA Module-Lattice-Based Digital Signature Algorithm

ML-KEM Module-Lattice-Based Key Encapsulation Mechanism

NIST National Institute of Standards and Technology

OAEP Optimal Asymmetric Encryption Padding

PQC Post-Quantum Cryptography

PSS Probabilistic Signature Scheme

RSA Rivest-Shamir-Adleman

SLH-DSA Stateless Hash-Based Digital Signature Algorithm

TLS Transport Layer Security

Symbols and Notations

Not applicable. This project does not introduce new mathematical notation; algorithm names follow the designations used in FIPS 203, FIPS 204 and FIPS 205.

ABSTRACT

The public-key cryptography that protects nearly all digital communication today rests on two mathematical problems — integer factorisation and discrete logarithms — that a sufficiently powerful quantum computer can solve efficiently using Shor's algorithm. RSA, Elliptic Curve Cryptography and Diffie-Hellman are therefore all on a path to obsolescence. The threat is not confined to the future: an adversary can record encrypted traffic now and decrypt it once such hardware exists, an approach known as harvest-now-decrypt-later. The National Institute of Standards and Technology responded by standardising replacements in August 2024, publishing ML-KEM in FIPS 203, ML-DSA in FIPS 204 and SLH-DSA in FIPS 205.

Standardising the algorithms, however, solved only half of the problem. Organisations still hold enormous bodies of existing software in which cryptographic calls are scattered across modules, buried inside third-party libraries, and entangled with application logic. Locating every vulnerable call, deciding what each one should become, rewriting it, and gaining confidence that nothing broke remains a slow and largely manual software-engineering task that demands cryptographic expertise most development teams do not have.

This project presents a working framework that automates that transition end to end for Python source code. Abstract-syntax-tree analysis locates quantum-vulnerable primitives and infers the role each one plays — key encapsulation, key exchange or digital signature — from evidence in the surrounding module. A rule-and-scoring engine then maps every finding to a specific NIST parameter set, taking into account the inferred purpose, a configurable target security category and strength signals recovered from the classical parameters, and records a written justification for each decision. Rewrite rules produce migrated copies of the source in which classical calls are redirected to a compatibility layer backed by genuine ML-KEM and ML-DSA implementations. Crucially, the framework then verifies its own output: it performs cryptographic round-trip and negative tests, and executes the migrated application.

The detector was evaluated on a purpose-built benchmark of twenty-nine Python modules containing thirty-eight expert-annotated cryptographic sites. Across the thirty-one sites falling within the stated scope it achieved precision of 1.00, recall of 1.00 and purpose classification accuracy of 1.00, raising no false alarms on six cryptographically clean modules. On a deliberately adversarial subset of seven sites reached through wrapper libraries, string parameters and dynamic dispatch, recall was 0.00, which usefully delimits where purely syntactic analysis ceases to be sufficient. Applied to three sample applications, the pipeline migrated all nine detected sites, and the rewritten RSA and ECDSA applications executed successfully under real post-quantum cryptography. Measured benchmarks quantify the cost of migration: ML-KEM-768 public keys are 4.03 times the size of RSA-2048 keys and ML-DSA-65 signatures are 46.6 times an ECDSA P-256 signature, while post-quantum key generation proved roughly eleven times faster.

CHAPTER 1

INTRODUCTION

1.1 Background

Modern digital infrastructure depends on public-key cryptography to a degree that is easy to underestimate. Every secure web session, every signed software update, every encrypted banking transaction and every authenticated government service relies on one of a small number of algorithms: RSA, Elliptic Curve Cryptography and Diffie-Hellman key agreement. These schemes have served well for three decades because the mathematical problems they depend upon, namely factoring large integers and computing discrete logarithms, are intractable for classical computers.

That intractability is a property of classical computation rather than of the problems themselves. In 1994 Peter Shor described a quantum algorithm that solves both problems in polynomial time [11]. A quantum computer able to sustain a sufficient number of coherent logical operations would therefore break RSA, ECC and Diffie-Hellman completely, not merely weaken them. Doubling key sizes offers no protection, because the speed-up is exponential rather than incremental.

Symmetric cryptography is in a far better position. Grover's algorithm offers only a quadratic improvement against symmetric ciphers and hash functions, which is answered adequately by increasing key lengths; AES-256 and SHA-384 remain appropriate choices. The migration problem is therefore confined to public-key cryptography. This narrows the target considerably, but the public-key layer is precisely the part most deeply entangled with application logic, protocol design and key management.

Recognising the scale of the eventual transition, NIST ran a multi-year standardisation process and published the resulting standards in August 2024: ML-KEM for key encapsulation in FIPS 203 [1], ML-DSA for digital signatures in FIPS 204 [2], and SLH-DSA, a conservative hash-based signature scheme, in FIPS 205 [3]. High-quality implementations of all three became available soon afterwards. From an algorithmic standpoint the question of what to migrate to has been answered.

1.2 Motivations

The motivation for this project arises from the gap between having standards and having migrated systems. Three considerations drove the work.

First, the threat is already active. Encrypted traffic intercepted today can be stored cheaply and decrypted years later once capable hardware exists. This harvest-now, decrypt-later strategy means that any information whose confidentiality must outlast the arrival of quantum computers is effectively exposed now [9]. Medical records, biometric and identity databases, diplomatic communications and long-dated financial contracts all fall into this category. Waiting until quantum computers are demonstrated is therefore not a viable strategy.

Second, migration is a software-engineering problem of substantial scale. Cryptographic

usage in a real codebase is rarely concentrated in one well-marked module. Key generation sits in one file, serialisation in another, and the actual encryption or signing operation in a third, often written years apart by developers who have since moved on. Identifying every occurrence by manual inspection is slow, and overlooking even a single site leaves the system vulnerable while creating a false impression of completion.

Third, correct replacement requires expertise that is scarce. Choosing between ML-KEM and ML-DSA is not interchangeable: a key-encapsulation mechanism cannot sign and a signature scheme cannot transport a key. Choosing the correct parameter set requires reasoning about the security level the deployment actually needs. Most development teams do not have a cryptographer available to make these judgements for every call site, and an incorrect substitution can produce code that executes successfully while providing no meaningful protection.

An automated framework that locates vulnerable cryptography, reasons about its purpose, selects a standardised replacement, performs the rewrite and then demonstrates that the result still functions correctly addresses all three concerns simultaneously. That is the system this project builds.

1.3 Scope of the Project

This Project-I phase delivers a complete and demonstrable pipeline rather than a partial prototype of every intended feature. The following capabilities are within scope and are implemented and evaluated in this report.

- Detection of RSA, Elliptic Curve Cryptography and Diffie-Hellman usage in Python source code through abstract-syntax-tree analysis, with a textual fallback for files that cannot be parsed.
- Project-wide static analysis producing an import map and a list of dependencies that introduce quantum-vulnerable cryptography.
- Application-context handling, allowing a target NIST security category and a preferred signature family to be supplied through a configuration file or command-line options.
- A rule-and-scoring recommendation engine that maps each detected site to a specific NIST parameter set and records a justification.
- Rule-based source migration producing modified copies of the project, leaving the original source untouched.
- Functional verification comprising key-encapsulation and signature round-trips, negative tests, and execution of the migrated application.
- Automated report generation in JSON, Markdown and HTML, including a performance and size benchmark measured at the time the report is produced.

The following are deliberately excluded from this phase and are discussed as future work in Chapter 8.

- Migration driven by a large language model. All rewriting in this phase is performed

by deterministic rules; sites that no rule matches are reported honestly as unmigrated rather than being transformed speculatively.

- Programming languages other than Python.
- An optimisation pass over migrated code. The module exists as a defined interface with no implementation, and does not claim to optimise anything.
- Protocol-level restructuring of Diffie-Hellman handshakes, which requires changing the surrounding protocol rather than substituting a call.
- Production deployment and any graphical interface beyond the command line.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

The literature relevant to this project divides into four strands, each addressing part of the migration problem while leaving the others open.

The first strand concerns cryptographic discovery. Guidance from the National Cybersecurity Center of Excellence identifies the construction of a cryptographic inventory as the necessary first step of any migration programme [4]. Work in this area produces a catalogue of where cryptography appears within an estate, which is valuable for planning. Discovery nevertheless stops at the inventory: establishing that a particular line invokes RSA does not indicate what should replace it, and produces no modified code.

The second strand comprises standards and migration guidance. The NIST preparatory white paper [7] and the FIPS standards themselves [1], [2], [3] establish both the destination and the principle of crypto-agility, under which cryptographic operations are isolated behind an interface so that algorithms may be exchanged without disturbing application code. This guidance is prescriptive about objectives and intentionally silent about implementation. It also has a structural limitation for existing software: a system already written with crypto-agility in mind is precisely the system that does not present a difficult migration, whereas the code that genuinely requires migration is the code that hard-wired its primitives.

The third strand consists of post-quantum implementations. The Open Quantum Safe project and its liboqs library made candidate and standardised algorithms broadly available well before standardisation concluded [10], and pure-Python implementations of ML-KEM and ML-DSA now exist [5], [6]. This body of work supplies the destination of a migration without addressing the process of arriving there.

The fourth strand applies large language models to source-to-source code transformation [8]. Reported results on isolated functions are encouraging, and the literature consistently attaches the same qualification: generated transformations require human verification, because a syntactically plausible patch is indistinguishable from a correct one without execution. In a cryptographic setting this qualification is particularly serious, since an incorrect transformation may yield software that runs normally while offering no protection.

Table 2.1: Summary of reviewed literature and its limitations

Strand	Representative work	Contribution	Limitation for this problem
Cryptographic discovery	NCCoE SP 1800-38 [4]	Establishes inventory as the first migration step	Produces a catalogue only; no recommendation and no code change
Standards and guidance	FIPS 203/204/205 [1]-[3]; NIST white paper [7]	Define the target algorithms and crypto-agility as a goal	Prescribe the destination, not a path for existing code
PQC implementations	liboqs [10]; kyber-py, dilithium-py [5], [6]	Make the standardised primitives usable in practice	Supply the destination without addressing migration
LLM code transformation	Chen et al. [8]	Demonstrate model-driven source transformation	Output requires verification; unverified crypto changes are unsafe

[Insert the additional papers identified during your own literature survey into Table 2.1 and discuss them in this section before submission.]

2.2 Research Gap

Reviewing these four strands together exposes a clear discontinuity. Discovery tools identify cryptography but do not recommend replacements. Standards specify replacements but do not transform code. Implementations provide the primitives but do not migrate anything. Transformation research changes code but does not verify the result. No existing system connects detection, decision-making, transformation and verification into a single workflow.

The verification stage deserves particular emphasis, because it is where the gap is widest. A migration that alters cryptographic code and reports success on the basis of the diff alone provides no assurance whatsoever. Cryptographic correctness is not visible by inspection; it must be demonstrated by executing the migrated system and confirming that keys agree, signatures validate, and tampered inputs are rejected. Closing the loop between transformation and verified execution is the gap this project addresses.

2.3 Objectives

The project set out to achieve the following objectives.

1. To design and implement a detection engine that locates quantum-vulnerable public-key cryptography in Python source code and reports the file, line, algorithm family and owning library for each occurrence.
2. To classify each detected occurrence by the cryptographic role it performs, distinguishing key encapsulation, key exchange and digital signature, since this determines which standard applies.
3. To develop a recommendation engine that selects a specific NIST parameter set for each finding on the basis of its purpose, a configurable target security category and strength signals recovered from the classical parameters, and that records a

justification for every decision.

4. To implement rule-based source migration that produces modified copies of a project in which classical calls are redirected to genuine post-quantum implementations, without modifying the original source.
5. To verify migrated output functionally through cryptographic round-trip tests, negative tests, and execution of the migrated application.
6. To generate automated reports in machine-readable and human-readable formats containing findings, recommendations, migration outcomes, verification results and a benchmark measured on the host machine.
7. To evaluate the detection engine quantitatively against a labelled benchmark and to report precision, recall and purpose-classification accuracy honestly, including the cases where the approach fails.

2.4 Problem Statement

Given a Python codebase of arbitrary size that uses classical public-key cryptography, produce an equivalent codebase in which those cryptographic operations are performed by NIST-standardised post-quantum algorithms, together with evidence that the transformed system remains functionally correct.

Formally, let the source project be a set of modules M. The detection problem is to compute the set F of all sites in M at which quantum-vulnerable public-key cryptography is invoked, maximising recall so that no vulnerable site is overlooked while maintaining precision so that developers are not burdened with spurious findings. The recommendation problem is to define a mapping from each finding f in F, together with an application context c, to a specific NIST parameter set, such that the chosen algorithm is of the correct cryptographic family for the purpose of f and meets the security category demanded by c. The migration problem is to produce a modified module set M' in which each site in F invokes its recommended replacement while the observable behaviour of the application is preserved. The verification problem is to establish, by execution rather than by inspection, that M' performs correct cryptographic operations.

A solution must further satisfy a safety property that distinguishes this problem from general code transformation: the system must never report a site as migrated unless a transformation rule actually applied to it, and must never report a migration as working unless the migrated code was executed successfully. Silent failure in a cryptographic migration is more dangerous than visible failure, because it leaves the operator believing a system is protected when it is not.

2.5 Project Plan

The work was organised into eight phases executed over the semester. Each module was implemented together with its unit tests before the next was begun, so that the pipeline remained demonstrable at every stage rather than only at the end.

Table 2.2: Project plan and phase-wise schedule

Phase	Activity	Deliverable	Status
1	Literature survey and problem formulation	Research gap, objectives, problem statement	Completed
2	Project skeleton, data model and CLI framework	Installable package with core data classes	Completed
3	Detection engine and pattern index	AST scanner with regex fallback and unit tests	Completed
4	PQC backend abstraction	Unified interface over liboqs and pure-Python implementations	Completed
5	Recommendation engine	Rule and scoring engine with justification output	Completed
6	Migration engine and compatibility shim	Rewrite rules producing executable migrated code	Completed
7	Verification, benchmarking and reporting	Round-trip tests, execution checks, three report formats	Completed
8	Evaluation and documentation	Labelled benchmark, measured results, project report	Completed

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional Requirements

The functional requirements define the behaviour the framework must exhibit. Each was validated by at least one automated test in the final test suite.

Table 3.1: Functional requirements

ID	Requirement	Verified by
FR1	The system shall recursively scan a directory of Python source files and report every occurrence of RSA, ECC or Diffie-Hellman usage.	Detector unit tests; benchmark
FR2	For each occurrence the system shall report the file path, line number, algorithm family, owning library and the matched construct.	Detector unit tests
FR3	The system shall classify each occurrence as key encapsulation, key exchange or digital signature.	Purpose classification benchmark
FR4	The system shall continue to analyse files that fail to parse, using a textual fallback.	Unparsable-file test
FR5	The system shall accept a target security category and preferred signature family from a configuration file or command-line option.	Context loader tests
FR6	The system shall recommend a specific NIST parameter set for every finding and record a justification.	Recommender tests
FR7	The system shall produce migrated copies of the source without modifying the original files.	Migration tests
FR8	The system shall not report a finding as migrated unless a rewrite rule actually applied to it.	Unsupported-pattern test
FR9	The system shall verify post-quantum operations by round-trip and negative tests, and shall execute migrated applications.	Verifier tests; end-to-end run

ID	Requirement	Verified by
FR10	The system shall generate reports in JSON, Markdown and HTML containing a benchmark measured on the host machine.	Reporter tests

3.1.2 Non-Functional Requirements

Table 3.2: Non-functional requirements

ID	Category	Requirement
NFR1	Correctness	Cryptographic claims must be established by execution rather than inspection; negative tests must accompany every positive test.
NFR2	Safety	The original source must never be modified; all output is written to a separate directory.
NFR3	Honesty	Unmigrated and partially migrated sites must be reported as such, with a stated reason.
NFR4	Portability	The framework must operate on any platform with Python 3.11 or later, and must not depend on a native build toolchain.
NFR5	Extensibility	Post-quantum operations must pass through a single interface so that backends can be exchanged without altering other modules.
NFR6	Auditability	Every recommendation must carry a human-readable justification, and every migrated file must record which rules were applied.
NFR7	Performance	A complete pipeline run on a small project must finish within approximately one minute on commodity hardware.
NFR8	Reproducibility	Reported measurements must be produced at run time on the host machine rather than stored as constants.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The project is technically feasible because each component depends on mature, available technology. Source analysis uses the abstract syntax tree module of the Python standard library, which exposes the same parser used by the interpreter and therefore matches the language exactly. Post-quantum operations are provided by existing implementations of the

standardised algorithms rather than by cryptographic primitives written for this project, which would be inappropriate for a student implementation and unnecessary given the quality of available libraries.

The principal technical risk identified at the outset was the availability of the post-quantum backend. The liboqs library offers the broadest algorithm coverage and the best performance but requires compiling a native library, which is an obstacle on some platforms. This risk was mitigated architecturally by defining a single backend interface and implementing two providers behind it: liboqs where it is available, and the pure-Python kyber-py and dilithium-py packages otherwise. Selection is automatic at run time. The consequence of using the pure-Python backend is slower execution, not reduced correctness, and the framework reports which backend produced any given measurement.

A second technical consideration is the structural mismatch between some classical and post-quantum primitives. RSA key transport maps cleanly onto ML-KEM encapsulation followed by symmetric encryption of the payload. Diffie-Hellman does not map cleanly, because its symmetry is essential to the protocol. Rather than attempting an unsound transformation, the design treats this case explicitly, which is discussed in Section 5.1 and Chapter 7.

3.2.2 Economic Feasibility

The project incurs no direct monetary cost. All software used is open source and freely available: Python and its standard library, the cryptography package for classical primitives and the symmetric layer, kyber-py and dilithium-py for post-quantum operations, pytest for testing, Jinja2 for report templating and Rich for terminal output. Development was carried out on personal computers already available to the team, and no cloud, hardware or licensing expenditure was required.

The economic case for the framework itself is stronger still. Manual cryptographic migration requires skilled engineering time, which is both expensive and scarce. Automating discovery, recommendation and the mechanical portion of rewriting reduces that effort substantially, and the measured benchmark allows an organisation to quantify the bandwidth and storage cost of migration before committing to it rather than discovering it in production.

3.2.3 Social Feasibility

The framework is designed for software developers and security engineers, and is delivered as a command-line tool of the kind such users employ routinely. No specialist cryptographic knowledge is required to operate it, which is deliberate: the shortage of cryptographic expertise is one of the motivations for the project. Every recommendation is accompanied by a plain-language justification so that a reviewer without a cryptography background can follow the reasoning, and reports are produced in HTML for non-technical stakeholders as well as in JSON for integration with other tooling.

The broader social benefit lies in protecting information that belongs to individuals. If the transition to post-quantum cryptography is delayed or performed incompletely, the data exposed is ultimately personal: health records, identity documents, financial histories and private communications. A tool that lowers the effort and expertise required to migrate correctly makes timely protection more likely, which is the alignment with Sustainable

Development Goal 9 on resilient infrastructure and Goal 16 on strong institutions discussed in Chapter 8.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.3: Hardware specification

Component	Specification used for development and evaluation
Processor	Intel 12th generation x86-64 processor
Memory	8 GB RAM (4 GB is sufficient to run the framework)
Storage	Approximately 500 MB for the environment and dependencies
Display	Standard laptop display; no graphics acceleration required
Network	Required only for the initial installation of dependencies

3.3.2 Software Specification

Table 3.4: Software specification

Component	Version	Purpose
Operating system	Windows 11	Development and evaluation platform; the framework is platform independent
Python	3.11.9	Implementation language and AST analysis
kyber-py	1.2.0	ML-KEM (FIPS 203) implementation
dilithium-py	1.4.0	ML-DSA (FIPS 204) implementation
liboqs-python	optional	Preferred backend where a native build is available
cryptography	50.0.1	Classical RSA, ECC and DH in sample applications; AES-GCM in the migrated layer
Click	8.5.0	Command-line interface
Rich	15.0.0	Formatted terminal output
Jinja2	3.1.6	HTML report templating
pytest	9.1.1	Automated test suite
Git	2.54	Version control

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 System Architecture

The framework is organised as a linear pipeline of seven stages, shown in Figure 4.1. Each stage is an independent module that communicates with its neighbours through explicit data structures rather than shared state. This arrangement was chosen for two reasons. It allows any stage to be exercised in isolation from the command line, which is valuable both for testing and for demonstration. More importantly, it makes it structurally impossible for a later stage to assert an outcome that an earlier stage did not produce: the reporter can only describe migrations that the migrator actually recorded.

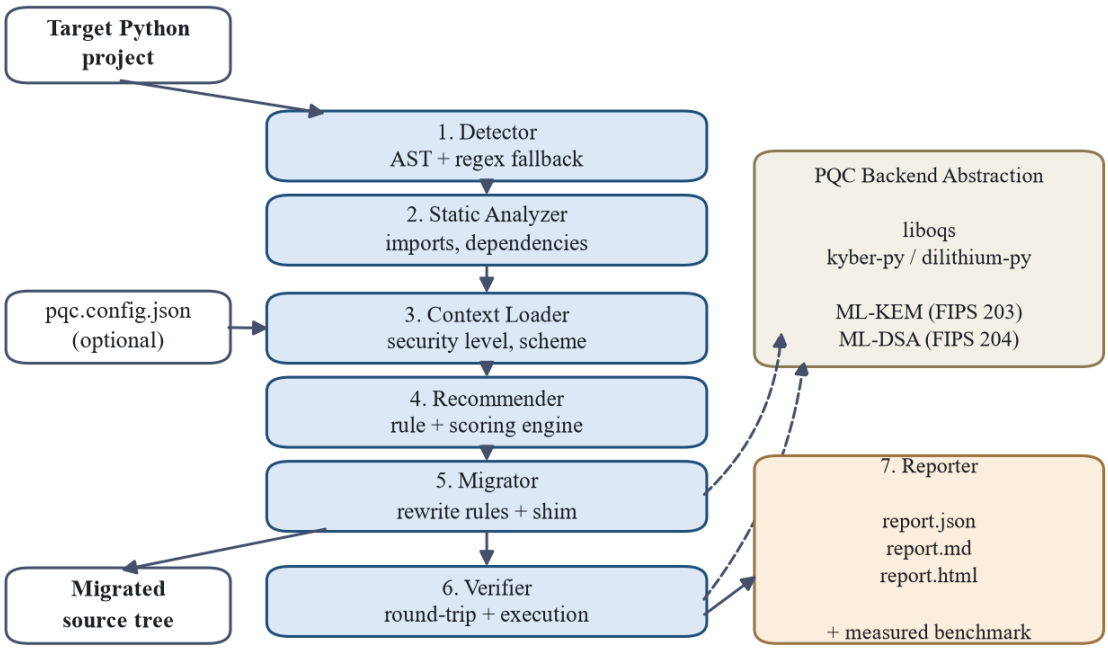


Figure 4.1: System architecture of the PQC migration framework

Four record types carry state along the pipeline. A Finding records one located occurrence of classical cryptography, comprising the file, line, algorithm family, inferred purpose, matched construct and owning library. A Recommendation pairs a finding with the selected parameter set, the governing FIPS document, the security category and a written justification. A MigrationResult records what happened to one finding, including which rules fired and whether human attention is still required. A Report aggregates all of these together with verification outcomes and benchmark measurements.

The post-quantum backend sits to one side of the pipeline rather than within it. Both the migrated code and the verifier obtain cryptographic operations through a single interface, so that the choice between liboqs and the pure-Python implementations is invisible to every other module. This is the crypto-agility principle applied to the framework's own design.

4.2 Design

4.2.1 Data Flow Diagram

Figure 4.2 presents the context diagram. The framework is treated as a single process exchanging data with one external entity, the developer or security engineer, who supplies a source path and an optional configuration file and receives migrated code and a migration report.

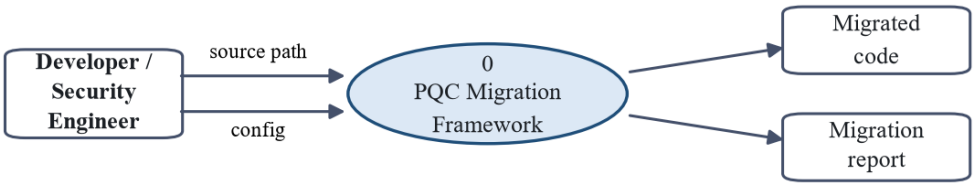


Figure 4.2: Data flow diagram - Level 0 (context diagram)

Figure 4.3 decomposes that process into its five principal sub-processes and the three data stores they consult. The pattern index supplies the detector with the constructs it recognises; the rewrite rules drive the migrator; and the post-quantum backend serves both migration and verification. The unidirectional flow of findings, recommendations, migration results and check results between sub-processes mirrors the module boundaries described in Section 4.1.

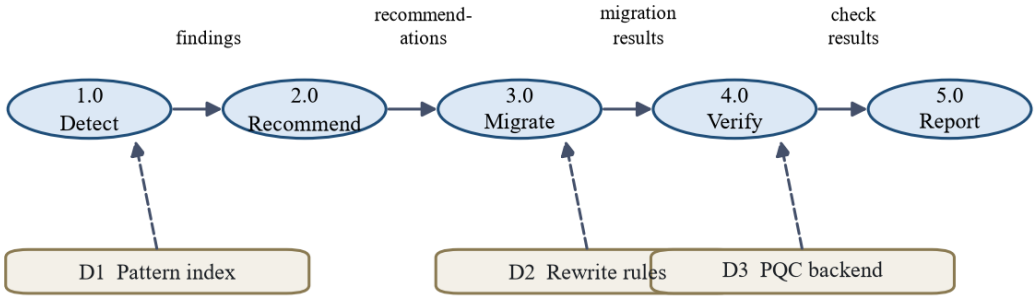


Figure 4.3: Data flow diagram - Level 1

4.2.2 Use Case Diagram

Figure 4.4 shows the use cases supported by the system and the actors that participate in them. A developer typically exercises scanning, recommendation, migration and verification while working on a codebase. A security engineer is more concerned with verification, execution of migrated applications and report generation, since these produce the evidence required to sign off a migration. The post-quantum backend participates as a supporting actor in every use case that performs a cryptographic operation.

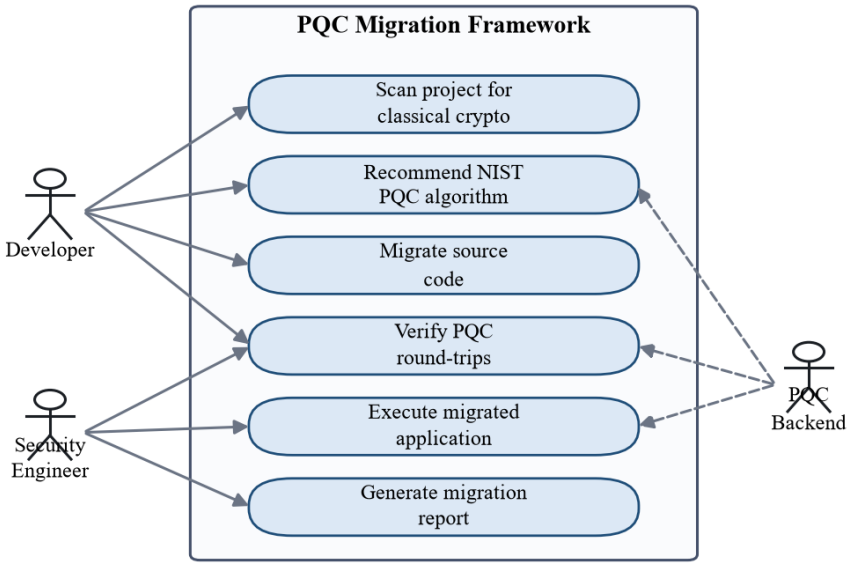


Figure 4.4: Use case diagram

4.2.3 Class Diagram

Figure 4.5 presents the core data model and the engines that operate on it. The four record types form a chain of composition: a Recommendation holds the Finding it concerns, a MigrationResult holds the Finding it acted upon, and a Report aggregates all of them. The Scanner, Migrator and PQCBackend classes are shown with their principal operations. Keeping the records separate from the engines allows every intermediate result to be serialised to JSON, which is what makes the pipeline inspectable at each stage.

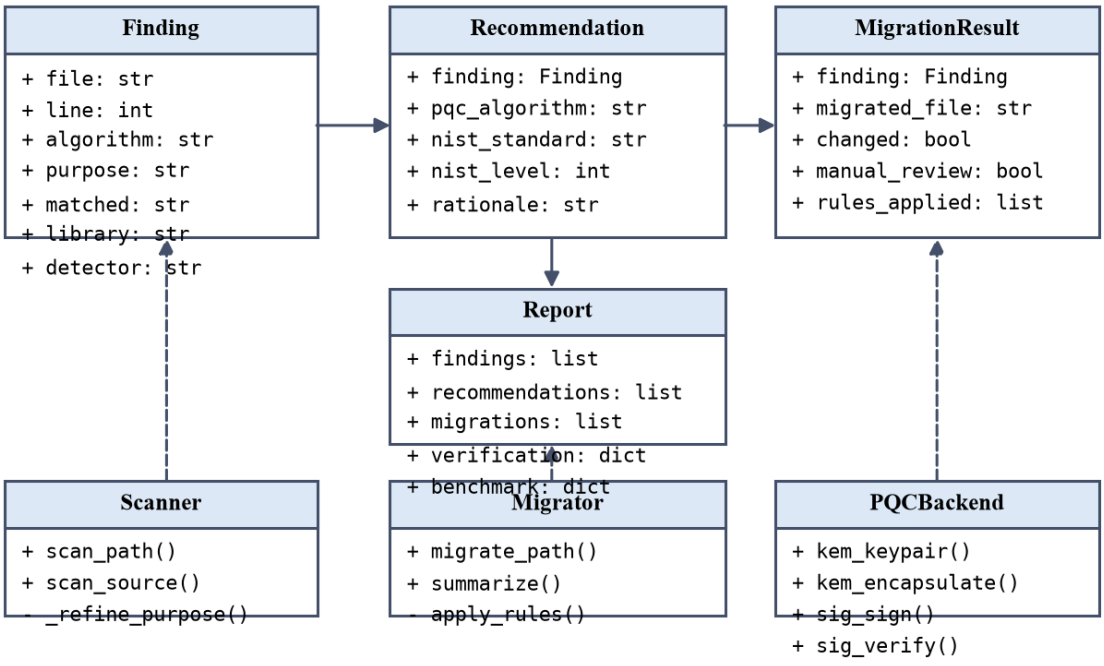


Figure 4.5: Class diagram of the core data model and engines

4.2.4 Sequence Diagram

Figure 4.6 traces a complete invocation of the run command. The command-line layer orchestrates the stages in order, passing the output of each into the next. Two self-directed messages are worth noting. The migrator applies its rewrite rules and copies the source tree internally, and the verifier performs both the cryptographic round-trips and the execution of the migrated application. The second of these is the step that distinguishes this design from one that merely proposes changes.

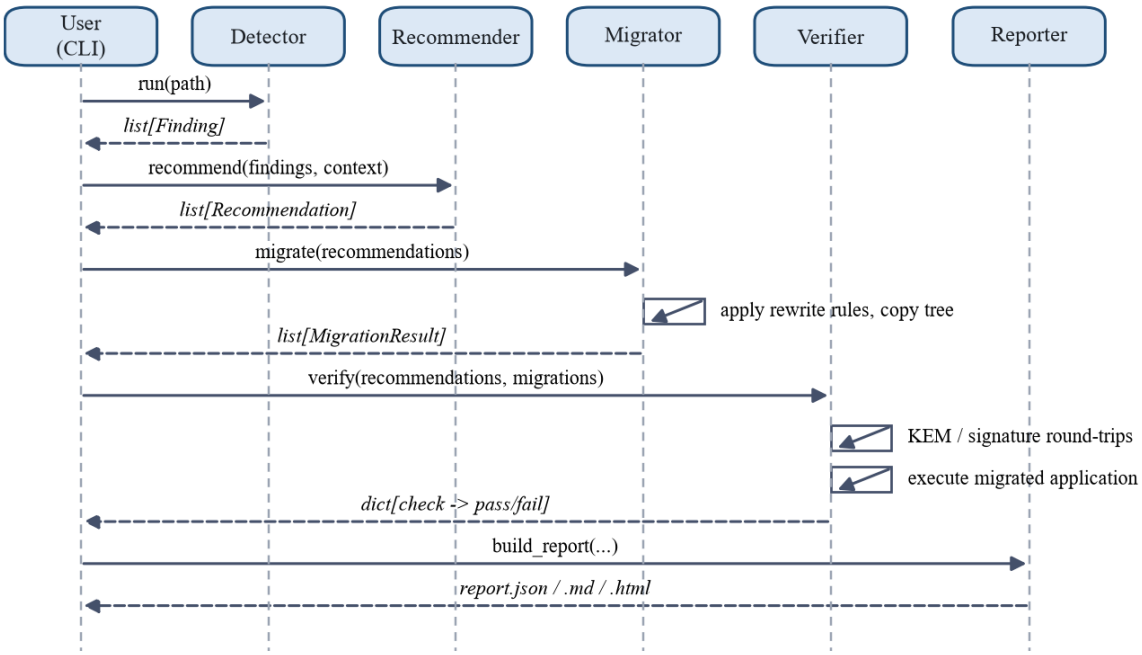


Figure 4.6: Sequence diagram for the end-to-end run command

CHAPTER 5

METHODOLOGY AND TESTING

5.1 Module Description

This section describes each module of the framework, the method it implements and the design decisions that shaped it.

5.1.1 Detection Module

The detector parses each source file into an abstract syntax tree and walks the tree looking for call and attribute nodes. Each node is flattened into a dotted name, such as rsa.generate_private_key, and matched against a pattern index covering RSA, elliptic-curve and Diffie-Hellman constructs across the cryptography, PyCryptodome and ecdsa libraries. Syntactic analysis rather than textual search is essential here: a search for the string RSA would simultaneously miss genuine usage that does not contain the term and flag comments that merely mention it.

Two refinements proved necessary during evaluation. The first is alias resolution. A module may import a primitive under a different name, and matching the written name alone would miss such usage entirely. The detector therefore records the import table of each module and, when a dotted name does not match directly, re-resolves its root through that table before retrying. The second concerns the textual fallback, which handles files that fail to parse. An earlier version ran this fallback on every file as a second opinion; evaluation showed that on parseable files it contributed nothing the tree walk had missed while matching cryptographic names inside docstrings, string constants and import statements. It is now used only when parsing fails.

5.1.2 Purpose Classification

Each finding is assigned one of three purposes, because the recommendation depends entirely upon it. Most constructs determine their own purpose: OAEP padding indicates key transport, PSS padding indicates signing, and an ECDH object indicates key agreement. Key generation does not. The pattern table must assign some default to rsa.generate_private_key and ec.generate_private_key, but the true purpose is a property of the surrounding module.

The detector therefore performs a second pass over the findings of each module and reassigns generation sites according to how the key is subsequently used. An RSA key in a module that only signs is reclassified as a signature key, and an elliptic-curve key in a module that only performs agreement is reclassified as key exchange. Both rules were added in response to misclassifications revealed by the benchmark described in Chapter 7.

5.1.3 Recommendation Engine

Recommendation combines three inputs. The purpose selects the family: key encapsulation and key exchange map to ML-KEM, signatures to ML-DSA, or to SLH-DSA where a deployment explicitly prefers hash-based signatures. The target security category, read from

an optional configuration file or supplied on the command line and defaulting to category 3, selects the parameter set within that family. A scoring pass then raises the target where the classical parameters imply a stronger requirement: an RSA modulus of 4096 bits or a P-521 curve both push the recommendation to category 5.

Every recommendation carries a justification naming the construct, the reason the family was chosen, the resulting category and any scoring signals that applied. This is not presentational. A migration proposal that a security engineer cannot audit is a proposal that will not be accepted in practice.

5.1.4 Migration Engine and Compatibility Layer

Migration never modifies its input. It writes a transformed copy of the source tree and leaves the original untouched, which makes the entire operation reversible by deletion. Transformations are expressed as small independent rules so that the framework can report exactly which rule fired on which file. Imports of supported names are redirected to a shim module, and key-generation calls have their argument lists replaced with the chosen algorithm identifier. Each rewritten file carries a header recording the rules applied and the algorithms selected.

The compatibility layer is what keeps the rewrites small. It exposes objects shaped like their classical counterparts, supporting generate_private_key, public_key, encrypt and decrypt, sign and verify, and private_bytes, so that call sites need not change, while the operations underneath are post-quantum. RSA key transport maps onto ML-KEM encapsulation with the payload protected by AES-256-GCM keyed from the resulting shared secret. Classical arguments that carry no post-quantum meaning, such as padding modes, are accepted and ignored.

Diffie-Hellman is handled differently, and deliberately so. Its symmetry is essential: both parties derive the same secret from a single exchange, whereas a key-encapsulation mechanism requires one party to encapsulate and the other to decapsulate. No compatibility layer can conceal this difference, because the surrounding protocol must change. The framework therefore migrates parameter generation, annotates each exchange site with the restructuring required, marks the file as needing manual review, and excludes it from execution during verification. It does not report such a file as successfully migrated. A tool that silently produced something plausible in this situation would be considerably more dangerous than one that stops and says so.

5.1.5 Post-Quantum Backend

All post-quantum operations pass through one interface exposing key generation, encapsulation, decapsulation, signing and verification. Two providers implement it: liboqs where available, and pure-Python ML-KEM and ML-DSA otherwise. Selection is automatic and may be overridden by an environment variable. Requesting an algorithm the active provider cannot supply, such as SLH-DSA on the pure-Python backend, raises an explicit error rather than silently substituting a different scheme.

5.1.6 Verification Module

Verification answers two separate questions. The first is whether the primitives work. For each recommended algorithm the framework performs a full round trip together with the corresponding negative tests: that a shared secret survives encapsulation and decapsulation and that a foreign key does not recover it; that a signature validates and that both a tampered message and a wrong public key are rejected. The negative tests carry most of the evidential weight, since a stage that confirms only successful paths cannot distinguish working cryptography from a stub that always reports success.

The second question is whether the migrated program works, and it is answered by executing that program and recording its exit status. This is the strongest practical evidence that a transformation preserved behaviour.

5.1.7 Reporting and Benchmarking Module

The reporter assembles findings, recommendations with justifications, migration outcomes, verification results and dependency analysis, and writes them in JSON for machine consumption, Markdown for documentation and HTML for presentation. The benchmark is measured when the report is generated rather than stored, so figures always reflect the machine and backend in use. Timings are reported as medians over repeated iterations to reduce the influence of scheduling noise.

Table 5.1: Module description summary

Module	Responsibility	Principal output
Detector	AST analysis with textual fallback; purpose classification	Findings
Static analyzer	Project walk, import map, vulnerable dependency list	Analysis summary
Context loader	Security category and signature preference	Application context
Recommender	Rule and scoring selection of a NIST parameter set	Recommendations
Migrator	Rewrite rules and compatibility shim	Migrated source tree
PQC backend	Unified ML-KEM and ML-DSA interface over two providers	Cryptographic operations
Verifier	Round-trip tests, negative tests, application execution	Check results
Reporter	Aggregation, rendering and live benchmarking	JSON, Markdown, HTML
Optimizer	Defined interface only; no implementation in this phase	None (Project-II)

5.2 Testing

Testing follows the principle that a cryptographic tool must demonstrate its claims rather than assert them. Three complementary levels of testing were applied.

Unit testing covers each module in isolation. The suite comprises seventy tests that execute in approximately three and a half seconds, which is short enough that the full suite is run after every change. Tests were written alongside each module rather than retrospectively, so that no module was considered complete until its tests passed.

Table 5.2: Test suite composition

Test module	Focus	Representative checks
test_detector.py	Detection and classification	AST matching, purpose reclassification, library attribution, regex fallback
test_recommender.py	Algorithm selection	Family and parameter selection per category, scoring, configuration handling
test_pqc_backends.py	Cryptographic correctness	ML-KEM and ML-DSA round-trips, exact key sizes, rejection of unsupported algorithms
test_migrator.py	Source transformation	Rewrite correctness, originals unmodified, execution of migrated applications
test_verifier.py	Verification behaviour	Positive and negative checks, handling of files needing manual review
test_reporter.py	Reporting and pipeline	Report completeness in three formats, full command-line pipeline
test_evaluation.py	Benchmark integrity	Reproduction of the reported precision, recall and purpose accuracy

Functional testing is performed by the verification stage itself and therefore runs on every invocation of the pipeline rather than only during development. Because it executes the migrated application, it exercises the transformed code in the same way a user would.

Benchmark testing establishes the cost of migration. Each operation is timed over twenty iterations and the median is reported, with key, ciphertext and signature sizes recorded as exact byte lengths. Classical baselines are measured in the same run on the same machine so that the comparison is fair rather than drawn from published figures.

Regression protection is provided by including the detection benchmark within the automated suite. The precision, recall and purpose-accuracy figures reported in Chapter 7 are asserted by a test, so a future change that degrades detection will fail the suite rather than pass unnoticed.

CHAPTER 6

PROJECT DEMONSTRATION

The framework is operated entirely from the command line. Each pipeline stage is available as an individual subcommand, and a composite command executes the whole pipeline. This section demonstrates both modes on the sample applications supplied with the project.

6.1 Command Line Interface

```
pqc-migrator scan       <path>          # list detected findings
pqc-migrator recommend  <path>          # findings with recommended PQC algorithms
pqc-migrator migrate    <path> -o out   # write migrated copies
pqc-migrator verify     <path>          # run PQC round-trip and negative checks
pqc-migrator report     <path> -o rep   # JSON, Markdown and HTML reports
pqc-migrator run        <path> -o out   # the complete pipeline
pqc-migrator backend                    # show the active PQC backend
```

Options allow the target security category to be overridden with --security-level, the signature family with --signature-scheme, and the benchmark iteration count with --runs.

6.2 Sample Applications

Three deliberately small but idiomatic applications accompany the framework and serve as demonstration targets. The first performs session-key transport using RSA-OAEP across two modules. The second signs release artifacts using ECDSA over the P-256 curve and carries its own configuration file specifying a target security category. The third performs a finite-field Diffie-Hellman handshake, and exists specifically to demonstrate the case the framework cannot fully automate.

6.3 End-to-End Demonstration

Executing the pipeline against the RSA application produces the following sequence of stages. The detector reports four findings across two files, all classified as key encapsulation. The recommender maps all four to ML-KEM-768 under FIPS 203 at security category 3. The migrator rewrites both files and reports four of four findings migrated. The verifier confirms the backend is available, performs the ML-KEM round trip and the foreign-key rejection test, and then executes the migrated application successfully, giving four passed checks of four. Finally the reporter writes three report files and prints the measured benchmark.

```
$ pqc-migrator run samples/app_rsa -o pqc_out/app_rsa

Target: samples/app_rsa  |  backend: purepy
Context: level 3, signatures ML-DSA (source: defaults)

1. Scan       4 findings across 2 files (RSA, purpose: kem)
2. Recommend  ML-KEM-768   FIPS 203   level 3   (all 4 findings)
3. Migrate    4/4 findings migrated, 2 files rewritten
4. Verify     backend_available                  PASS
              kem_roundtrip[ML-KEM-768]          PASS
              kem_rejects_foreign_key[ML-KEM-768] PASS
              migrated_app_runs                  PASS
5. Report     report.json / report.md / report.html
```

```
Verification: 4/4 checks passed
```

Running the same command against the ECDSA application demonstrates that the pipeline is not specialised to one example. The detector reports four findings, all classified as signatures; the recommender selects ML-DSA-65 under FIPS 204, a different algorithm from a different standard; the security category is read from the application's own configuration file rather than from the default; and verification performs signature round-trip, tampered-message and wrong-key checks before executing the migrated application, giving five passed checks of five.

6.4 Migration Output

The effect of migration is most clearly seen by comparing a source file before and after. The original key-generation function reads as follows.

```
from cryptography.hazmat.primitives.asymmetric import rsa

def generate_keypair():
    private_key = rsa.generate_private_key(
        public_exponent=65537, key_size=KEY_SIZE)
    return private_key, private_key.public_key()
```

After migration the same function appears in the output directory as:

```
# --- migrated by pqc-migrator (rule-based, Project-I) ---
# rules applied: cryptography-imports-to-shim (x2), rsa-keygen-to-ml-kem (x1)
# post-quantum algorithms: ML-KEM-768 / ML-DSA-65

from pqc_shim import rsa

def generate_keypair():
    private_key = rsa.generate_private_key(algorithm="ML-KEM-768")
    return private_key, private_key.public_key()
```

The call site retains its shape, so dependent code continues to function, while the operation underneath is now ML-KEM. The header records precisely which rules produced the change, which is what makes the transformation auditable.

CHAPTER 7

RESULT AND DISCUSSION

This chapter reports the measured performance of the framework. Four questions are addressed: how accurately the detector locates vulnerable cryptography, whether it correctly infers the purpose of that cryptography, whether the pipeline produces migrated code that actually runs, and what migration costs. All measurements were taken on an Intel 12th-generation x86-64 laptop running Windows 11 and CPython 3.11.9, using the pure-Python backend.

7.1 Detection Accuracy

No standard benchmark exists for this task, so one was constructed. It comprises twenty-nine Python modules in three groups. Eighteen in-scope modules use RSA, ECC or Diffie-Hellman directly through the three targeted libraries. Five extended modules contain genuinely vulnerable cryptography reached indirectly. Six clean modules contain no vulnerable public-key cryptography at all; this group is essential, because a detector evaluated only on positive cases cannot be shown to be precise.

Ground truth is embedded in the corpus itself. Every line that a cryptographer would flag carries a trailing annotation naming the expected algorithm family, purpose and scope. Because the tree walk ignores comments and the textual fallback strips them, these labels cannot influence the measurement. The corpus contains thirty-eight annotated sites, of which thirty-one are in scope and seven are extended.

Table 7.1: Detection benchmark composition

Group	Modules	Annotated sites	Purpose of the group
In-scope	18	31	RSA, ECC and DH reached directly through targeted libraries
Extended	5	7	Vulnerable cryptography reached indirectly; adversarial by design
Clean	6	0	No vulnerable public-key cryptography; measures false alarms
Total	29	38	

A detection is counted at the granularity of file, line and algorithm family, so that a tool is neither rewarded nor penalised for how finely it decomposes a single site. Precision is computed over every flag raised against annotated sites of either scope, so that detecting an extended site is never penalised, and recall is reported separately for each scope.

Table 7.2: Detection performance

Metric	Value
Flags raised	31
Precision (all flags)	1.000
Recall, in-scope (31 sites)	1.000
F1, in-scope	1.000
Recall, extended (7 sites)	0.000
Clean modules producing false alarms	0 of 6

Table 7.3: In-scope detection performance by algorithm family

Algorithm family	Sites	Precision	Recall	F1
RSA	15	1.000	1.000	1.000
ECC	14	1.000	1.000	1.000
Diffie-Hellman	2	1.000	1.000	1.000

Within its stated scope the detector missed nothing and raised no false alarms, including on modules constructed specifically to mislead it. One clean module contains identifiers such as ECDSALogFormatter and generate_private_key_id that resemble cryptography without being cryptography; another discusses rsa.generate_private_key and dh.generate_parameters in its docstrings and string constants without ever calling them. Neither produced a finding.

These figures should be read together with the limitations stated in Section 7.5. The benchmark was authored by the same team that built the detector and it is small. It establishes that the pattern index, alias resolution and purpose refinement behave as designed on idiomatic code; it does not establish that detection is a solved problem.

The extended set is in fact the more informative result. Recall of 0.000 across all seven adversarial sites is not a defect to be apologised for but a measurement of where syntactic analysis ends. The misses fall into three categories. In the first, the algorithm is selected by a value rather than by an interface: a call of the form jwt.encode with the argument RS256 performs RSA signing, yet nothing in the call graph names RSA. In the second, cryptography is reached through a wrapper library outside the declared scope, such as paramiko or pyOpenSSL; these are the cheapest to address, requiring only an extension of the pattern index. In the third, indirection or dynamic dispatch defeats name-based matching, which requires dataflow analysis or the semantic reasoning that motivates the model-assisted component planned for Project-II.

Building the benchmark was not solely a measurement exercise; it changed the detector. The first run produced precision of 0.871 and recall of 0.964. Attributing each error identified three genuine defects: the textual fallback firing on string literals and import lines, accounting for four false positives; aliased imports being invisible to the pattern index, accounting for one false negative; and elliptic-curve keys used for agreement being misclassified as signing keys. Alias resolution, the restriction of the fallback to unparseable files, and the elliptic-curve purpose refinement all originate from this analysis. A coverage gap in Ed25519 and X25519, which are elliptic-curve schemes equally broken by Shor's algorithm, was closed at the same time. The figures reported above are those obtained after these corrections, and the benchmark now runs within the automated test suite so that they cannot regress unnoticed.

7.2 Purpose Classification

Purpose was evaluated on the thirty-one correctly located in-scope sites against the expert annotation. All thirty-one were classified correctly, giving an accuracy of 1.000. This includes the two cases for which the per-module refinement exists: an RSA key used exclusively for PSS signing, and an elliptic-curve key used exclusively for ECDH agreement. Before the refinement was added, the latter was classified as a signature key, which would have produced an ML-DSA recommendation for a key-exchange site. Such a recommendation is not merely suboptimal but unusable, since a signature scheme cannot perform key agreement.

7.3 End-to-End Migration Results

Table 7.4 summarises the complete pipeline applied to the three sample applications.

Table 7.4: End-to-end pipeline results on the sample applications

Application	Sites	Recommendation	Migrated	Manual review	Verification
RSA key transport	4 (2 files)	ML-KEM-768, FIPS 203	4 / 4	0	4 / 4 passed
ECDSA signing	4 (1 file)	ML-DSA-65, FIPS 204	4 / 4	0	5 / 5 passed
DH handshake	1 (1 file)	ML-KEM-768, FIPS 203	1 / 1	1	3 / 3 passed

Two observations matter more than the counts themselves. First, the two applications whose migrations completed were executed after rewriting, and both ran successfully under genuine post-quantum cryptography: the RSA application established a session key through ML-KEM encapsulation, and the ECDSA application produced and verified an ML-DSA signature. This is the distinction between a proposed patch and a verified migration.

Second, the two applications received different recommendations drawn from different FIPS standards, determined by the inferred purpose rather than by any per-project configuration, while the ECDSA project additionally had its security category read from its own configuration file. The pipeline is therefore general rather than specialised to a single example.

The Diffie-Hellman application behaves as designed. Parameter generation was migrated, the exchange site was annotated with the restructuring required, the file was marked as needing manual review, and it was excluded from execution. Its three verification checks cover the backend and the ML-KEM primitives rather than the application, and the report states this explicitly rather than inflating the result.

7.4 Cost Analysis

Migration is frequently discussed in terms of computational cost. The measurements show that the dominant cost is in fact size. Figure 7.1 and Table 7.5 compare key and output sizes; Figure 7.2 and Table 7.6 compare timings.

Table 7.5: Measured key and output sizes (bytes)

Scheme	Category	Public key	Secret key	Ciphertext / signature
ML-KEM-512	1	800	1,632	768
ML-KEM-768	3	1,184	2,400	1,088
ML-KEM-1024	5	1,568	3,168	1,568
ML-DSA-44	2	1,312	2,560	2,420
ML-DSA-65	3	1,952	4,032	3,309
ML-DSA-87	5	2,592	4,896	4,627
RSA-2048 (classical)	-	294	-	256
ECDSA P-256 (classical)	-	91	-	71

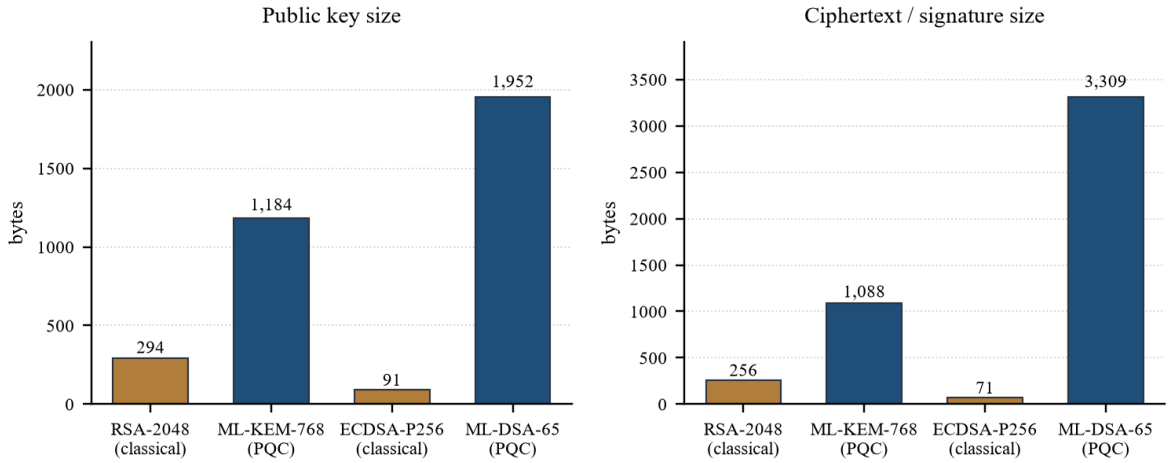


Figure 7.1: Classical versus post-quantum key and output sizes

ML-KEM-768 public keys are 4.03 times the size of RSA-2048 public keys and its ciphertexts 4.25 times the size. ML-DSA-65 signatures are 46.6 times the size of an ECDSA P-256 signature, and its public keys 21.4 times. These ratios are fixed by the standards and are independent of implementation. For a protocol that transmits certificate chains or signs large numbers of small messages, this is the dominant consideration in planning a migration.

Table 7.6: Measured performance of classical and post-quantum schemes

Scheme	Key generation (ms)	Primary operation (ms)	Inverse operation (ms)
ML-KEM-768	3.12	encapsulate 3.54	decapsulate 5.15
RSA-2048	33.71	encrypt 0.02	decrypt 0.49
ML-DSA-65	7.60	sign 34.72	verify 10.19
ECDSA P-256	0.014	sign 0.032	verify 0.069

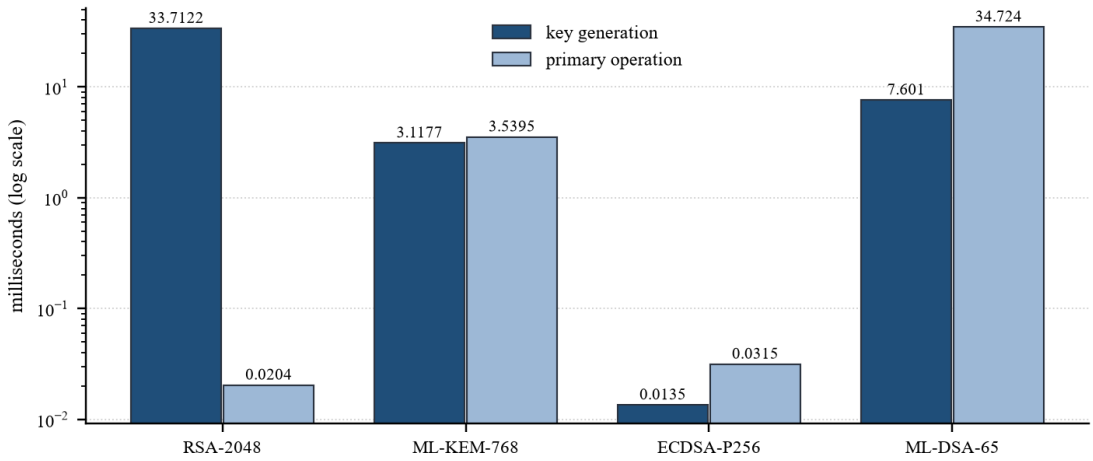


Figure 7.2: Classical versus post-quantum operation timings

ML-KEM-768 key generation is approximately eleven times faster than RSA-2048 key generation. This advantage is structural and independent of implementation, since lattice key generation involves no search for prime numbers. The ML-DSA signing figure of 34.72 milliseconds, by contrast, reflects a pure-Python implementation; optimised native implementations perform the same operation in well under a millisecond. The measured number is reported rather than a more favourable one, and is identified as a property of the backend rather than of the algorithm.

RSA key generation also shows noticeable variation between runs, observed between approximately 29 and 43 milliseconds, because it searches randomly for prime numbers. This variability is one reason the framework measures at report time rather than shipping a table of constants.

7.5 Discussion and Limitations

Several threats to the validity of these results should be acknowledged. The most significant concerns internal validity: the benchmark was constructed by the same team that implemented the detector. Although sites were labelled according to cryptographic role rather than according to what the tool reports, and although adversarial and clean modules were included deliberately, a self-authored benchmark cannot exclude the possibility that the corpus reflects assumptions already embedded in the implementation. Independent evaluation is required before the in-scope figures should be regarded as characteristic.

With respect to external validity, thirty-eight annotated sites across twenty-nine modules is a small sample, and the modules are short and self-contained. Production codebases are larger, less idiomatic, and contain cryptography reached through layers of internal abstraction that the extended set only begins to represent. The in-scope results should therefore be read as establishing correct behaviour on clear cases rather than as a projection of performance in the field.

A construct-validity limitation also applies to verification. Executing a migrated program establishes that it runs and that its cryptographic round trips succeed. It does not establish that the migrated system's security properties match those of the original in a protocol sense, since key lifetimes, session binding and authentication context lie outside what execution alone can

confirm.

The implementation itself carries known limitations. It handles Python only. Rewrite rules cover the cryptography library, while PyCryptodome and ecdsa usage is detected and reported honestly as unmigrated rather than being silently skipped. The optimisation stage exists as an interface without an implementation. SLH-DSA can be recommended but requires the liboqs backend in order to execute. Most significantly, migration is purely rule-driven: code whose shape no rule matches is copied unchanged and reported as such.

CHAPTER 8

CONCLUSION

Post-quantum migration is often framed as a cryptographic problem. The work reported here suggests it is better understood as a software-engineering problem subject to a cryptographic correctness criterion. The algorithms exist and have been standardised; what has been missing is a dependable method for moving a specific body of existing code from where it is to where it must be, accompanied by evidence that nothing broke in the process.

This project designed and implemented a framework that treats that transition as a single verified pipeline comprising detection, purpose inference, context-sensitive recommendation, rule-based rewriting, functional verification and reporting. On a purpose-built labelled benchmark the detector achieved precision of 1.00 and recall of 1.00 across thirty-one in-scope sites, with purpose classification accuracy of 1.00 and no false alarms on six clean modules, while recording recall of 0.00 on a deliberately adversarial set that marks the boundary of syntactic analysis. Applied end to end, the pipeline migrated all nine detected sites across three applications, and the two applications it migrated completely were executed successfully under genuine ML-KEM and ML-DSA. Measurement of the resulting overhead showed that the practical cost of migration lies in transmitted size rather than in computation.

All seven objectives stated in Section 2.3 were met. The detection, recommendation, migration, verification and reporting modules are implemented and demonstrable; the evaluation is quantitative; and the limitations are reported rather than concealed.

8.1 Sustainable Development Goals

The project aligns primarily with Sustainable Development Goal 9, Industry, Innovation and Infrastructure. Digital infrastructure in banking, healthcare, identity and government services rests on cryptography that quantum computing will break, and Target 9.1 concerns the development of reliable and resilient infrastructure. By automating the discovery and replacement of vulnerable cryptography, the framework lowers the barrier to making that infrastructure quantum-resilient, which also supports Target 9.5 on upgrading technological capability.

A secondary alignment exists with Goal 16, Peace, Justice and Strong Institutions, particularly Target 16.10 concerning the protection of fundamental freedoms. Cryptographic failure at scale exposes personal data: medical histories, identity records, financial information and private communications. Because the harvest-now-decrypt-later strategy means such data is already being collected, migrating in advance is a direct protection of individual privacy.

8.2 Future Work

Three directions follow from the measurements rather than from speculation.

The first is model-assisted transformation for code that rules cannot reach. The extended-set result quantifies the need precisely: seven sites in three distinct categories that name-based matching cannot see. A large language model is a plausible instrument for two of those

categories, namely semantic parameters and wrapper libraries, and the framework is already structured to accommodate one, with the rule engine attempted first and the model invoked only on what remains. The critical design point is that the verification stage does not move: model-generated code would be subject to exactly the same round-trip tests and execution requirement as rule-generated code. This converts the standard objection to generative code transformation, that its output cannot be trusted without verification, into a tractable engineering requirement.

The second is protocol-level restructuring. The Diffie-Hellman case requires more than a better rewrite rule; the surrounding protocol must be transformed into an encapsulate and decapsulate exchange. This is a program transformation over control flow rather than a substitution, and is a natural candidate for the same model-assisted approach.

The third is broader coverage and independent evaluation. Extending the pattern index to wrapper libraries such as paramiko and pyOpenSSL is inexpensive and would immediately close part of the extended set. Supporting languages beyond Python requires per-language parsing but no architectural change. Most importantly, the framework should be run against real repositories with independently established ground truth before any claim about effectiveness in the field is justified. Alongside these, the optimisation module defined in the architecture remains to be implemented, as does a graphical interface for users who do not work at the command line.

CHAPTER 9

REFERENCES

References are listed in order of recency, with the most recently published source first.

[1] National Institute of Standards and Technology, Module-Lattice-Based Key-Encapsulation Mechanism Standard, FIPS 203, Gaithersburg, MD, USA, Aug. 2024.

[2] National Institute of Standards and Technology, Module-Lattice-Based Digital Signature Standard, FIPS 204, Gaithersburg, MD, USA, Aug. 2024.

[3] National Institute of Standards and Technology, Stateless Hash-Based Digital Signature Standard, FIPS 205, Gaithersburg, MD, USA, Aug. 2024.

[4] National Cybersecurity Center of Excellence, Migration to Post-Quantum Cryptography, NIST Special Publication 1800-38, National Institute of Standards and Technology. [Confirm the volume and revision date cited.]

[5] G. Pope, "kyber-py: A pure-Python implementation of ML-KEM (FIPS 203)," software repository, version 1.2.0.

[6] G. Pope, "dilithium-py: A pure-Python implementation of ML-DSA (FIPS 204)," software repository, version 1.4.0.

[7] W. Barker, W. Polk and M. Souppaya, Getting Ready for Post-Quantum Cryptography: Exploring Challenges Associated with Adopting and Using Post-Quantum Cryptographic Algorithms, NIST Cybersecurity White Paper, 2021.

[8] M. Chen et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[9] M. Mosca, "Cybersecurity in an era with quantum computers: Will we be ready?," IEEE Security & Privacy, vol. 16, no. 5, pp. 38-41, 2018.

[10] D. Stebila and M. Mosca, "Post-quantum key exchange for the Internet and the Open Quantum Safe project," in Selected Areas in Cryptography (SAC 2016), Lecture Notes in Computer Science, vol. 10532, pp. 14-37.

[11] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," SIAM Journal on Computing, vol. 26, no. 5, pp. 1484-1509, 1997.

APPENDIX A – SAMPLE CODE

The following extracts show the core of each stage of the pipeline. The complete source, the three sample applications, the labelled detection benchmark and the evaluation harness are maintained in the project repository.

A.1 Detection: resolving a construct to a pattern

Source file: pqc_migrator/detector/scanner.py

```
    def _canonical(self, dotted: str) -> str:
        """Expand a local alias back to the module it was imported from."""
        root, _, rest = dotted.partition(".")
        origin = self.aliases.get(root)
        if not origin:
            return ""
        return f"{origin}.{rest}" if rest else origin

    def _record(self, node: ast.AST, dotted: str) -> None:
        pattern = P.lookup(dotted)
        if pattern is None:
            canonical = self._canonical(dotted)
            pattern = P.lookup(canonical) if canonical else None
        if pattern is None:
            return
        line = getattr(node, "lineno", 0)
        self.findings.append(Finding(
            file=self.file, line=line,
            algorithm=pattern.algorithm, purpose=pattern.purpose,
            matched=pattern.name,
            library=_library_for(dotted, self.aliases, pattern.library)))
```

A.2 Purpose refinement from surrounding usage

Source file: pqc_migrator/detector/scanner.py

```
    def _refine_ecc_purpose(file_findings: list[Finding]) -> None:
        """An EC key used only for agreement is key exchange, not signing."""
        ecc = [f for f in file_findings if f.algorithm == ALG_ECC]
        if not ecc:
            return
        agrees = any(f.purpose == PURPOSE_KEY_EXCHANGE for f in ecc)
        signs = any(f.matched in {"ec.ECDSA",
                                  "Ed25519PrivateKey.generate"} for f in ecc)
        if agrees and not signs:
            for finding in ecc:
                if finding.matched.startswith(("ec.generate", "ec.SECP")):
                    finding.purpose = PURPOSE_KEY_EXCHANGE
```

A.3 Recommendation with scoring and justification

Source file: pqc_migrator/recommender/recommend.py

```
    def recommend(finding, context=None):
        context = context or AppContext()
        level, reasons = _score_level(finding, context.security_level)

        if finding.purpose in (PURPOSE_KEM, PURPOSE_KEY_EXCHANGE):
            algorithm = KEM_BY_LEVEL[level]
        elif context.prefer_signature_scheme == "SLH-DSA":
            algorithm = SLH_DSA_BY_LEVEL[level]
        else:
            algorithm = ML_DSA_BY_LEVEL[level]

        return Recommendation(
```

```
            finding=finding, pqc_algorithm=algorithm,
            nist_standard=standard_for(algorithm), nist_level=level,
            rationale="; ".join(rationale_parts) + ".")
```

A.4 The compatibility layer: RSA key transport onto ML-KEM

Source file: pqc_migrator/migrator/pqc_shim.py

```
class KEMPublicKey:
    def encrypt(self, plaintext: bytes, padding=None) -> bytes:
        """ML-KEM encapsulate, then AES-256-GCM the payload."""
        ciphertext, shared_secret = backends.kem_encapsulate(
            self.algorithm, self.key_bytes)
        envelope = _aes_gcm_encrypt(shared_secret, plaintext)
        return struct.pack(">I", len(ciphertext)) + ciphertext + envelope

class KEMPrivateKey:
    def decrypt(self, blob: bytes, padding=None) -> bytes:
        (ct_len,) = struct.unpack(">I", blob[:4])
        ciphertext = blob[4:4 + ct_len]
        shared_secret = backends.kem_decapsulate(
            self.algorithm, self.secret_key, ciphertext)
        return _aes_gcm_decrypt(shared_secret, blob[4 + ct_len:])

    def exchange(self, peer_public_key) -> bytes:
        raise NotImplementedError(
            "ML-KEM is not a drop-in for DH.exchange(): the handshake must "
            "become an explicit encapsulate / decapsulate flow.")
```

A.5 Verification, including the negative checks

Source file: pqc_migrator/verifier/verify.py

```
def verify_signature(algorithm, message=DEFAULT_MESSAGE):
    results = {}
    public_key, secret_key = backends.sig_keypair(algorithm)
    signature = backends.sig_sign(algorithm, secret_key, message)

    results[f"signature_roundtrip[{algorithm}]"] = \
        backends.sig_verify(algorithm, public_key, message, signature)
    results[f"signature_rejects_tampered_message[{algorithm}]"] = not \
        backends.sig_verify(algorithm, public_key, message + b"!", signature)
    other_public, _ = backends.sig_keypair(algorithm)
    results[f"signature_rejects_wrong_key[{algorithm}]"] = not \
        backends.sig_verify(algorithm, other_public, message, signature)
    return results
```

A.6 Evaluation harness: scoring the detector

Source file: evaluation/evaluate.py

```
legitimate = detected & expected_all
spurious   = detected - expected_all
core_found = core & detected
core_missed = core - detected

precision       = _div(len(legitimate), len(detected))
core_recall     = _div(len(core_found), len(core))
extended_recall = _div(len(extended & detected), len(extended))
```